

Kapittel 13

Lagrede programmer

Læringsmål

- ☐ Kjenne til bruksområder for triggere og lagrede rutiner.
- ☐ Forstå hvordan triggere og lagrede rutiner blir utført.
- ☐ Kunne programmere enkle triggere og lagrede rutiner i PostgreSQL.

Lagrede rutiner

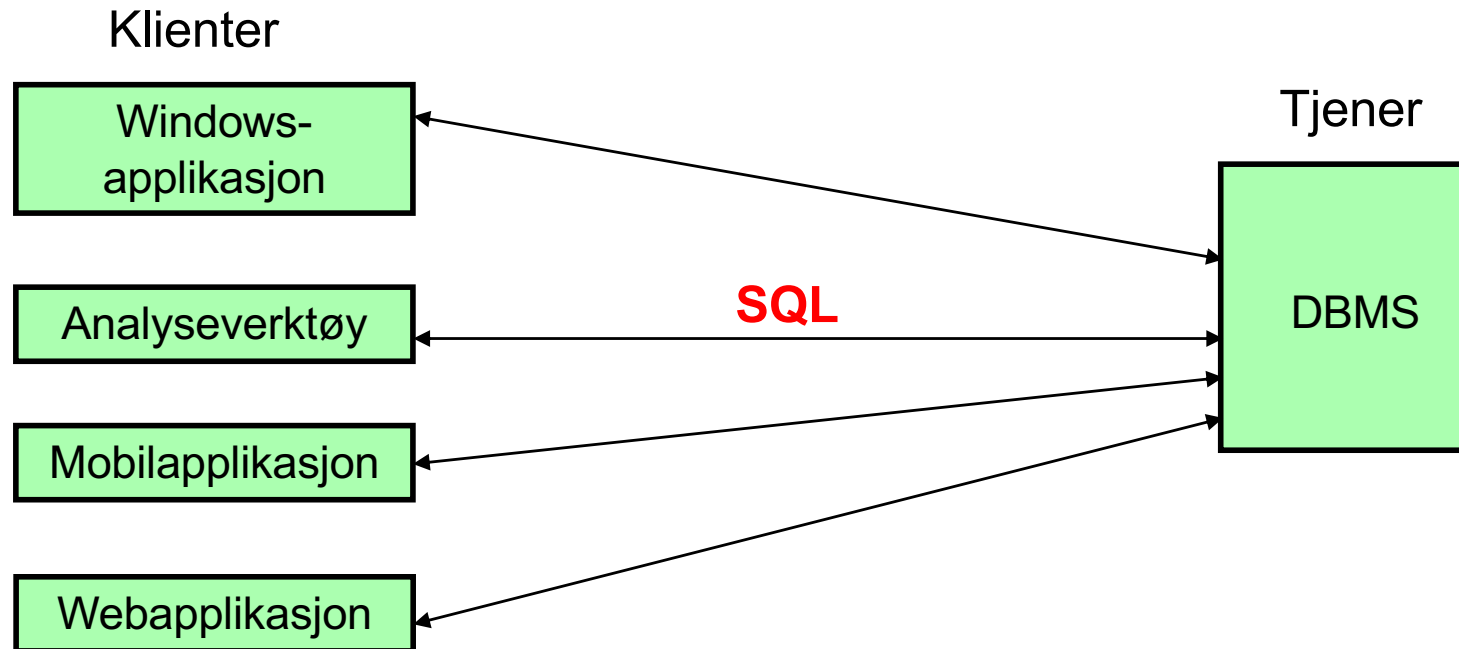
Definisjon av lagrede rutiner

- Lagrede prosedyrer
- Lagrede funksjoner
- Parameteroverføring: IN, OUT, INOUT
- Returverdier

SQL-spørringer i lagrede rutiner

- INSERT, UPDATE, DELETE
- SELECT INTO
- CURSOR

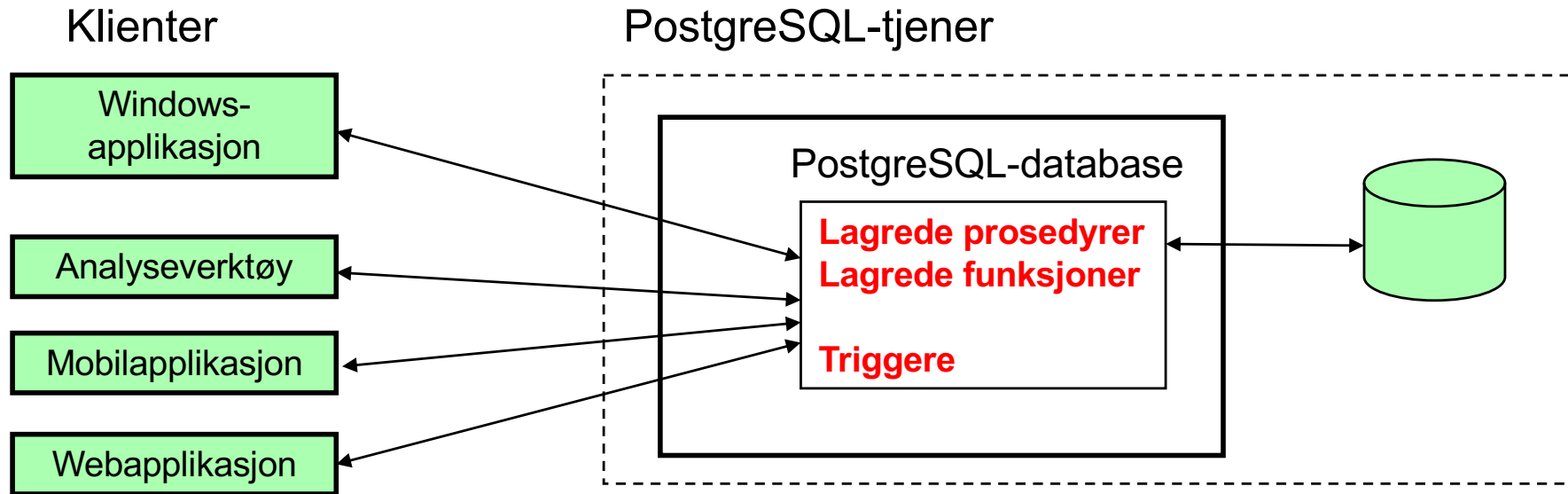
«Tradisjonell» klient / tjener - arkitektur



All funksjonalitet / logikk er programmert i klienten

- DBMS brukes kun til lagring / henting av data med SQL
- Samme metode (algoritme) må ofte lages i flere klientprogrammer
- Endringer i databasestruktur krever endringer i flere klientprogrammer

Databaseprogrammer i Databaseserveren



Gjenbruk og automatisering

- Lagrede prosedyrer kjører på tjeneren og kan kalles fra flere klienter
- Kan redusere nettverkstrafikk
- Triggere aktiveres automatisk ved endringer i databasen
- Kan kombineres med bruk av SQL fra klienten

Lagrede rutiner

Lagret **prosedyre**

- Registrere ny ansatt
- Data om den ansatte er parametre
- Kontrollerer inndata og setter inn ny rad hvis alt er ok
- Kan sende tilbake eventuell feilmelding i en ut-parameter

Lagret **funksjon**

- Gjennomsnittspris for varer i en kategori
- Kategorinummer er parameter
- Prisen blir returverdi

3 typer parameteroverføring

IN - parametre

- Er standard og svarer til verdioverføring i Java/C
- Benyttes for å sende verdier inn til prosedyren

OUT - parametre

- Benyttes for å returnere verdier fra prosedyren
- Hensiktsmessig hvis man vil returnere mer enn én verdi

IN OUT er en kombinasjon av IN og OUT

- Variabelverdien sendes inn i prosedyren og kan endres der
- Ev. endret verdi sendes tilbake fra prosedyren

Merk:

- Variable som IN-parametre endrer ikke verdi som følge av prosedyrekallet
- Variable som OUT og IN OUT parametre kan endre verdi

Definisjon og kall av lagret prosedyre

Først et litt atypisk eksempel som ikke bruker databasen, men som illustrerer inn- og ut-parametre godt.

```
CREATE PROCEDURE intdiv(  
    IN  x INT,  
    IN  y INT,  
    OUT d INT,  
    OUT r INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    d := x / y;  
    r := x % y;  
END;  
$$;
```

Prosedyre kall
(fra en annen prosedyre)

```
DECLARE svar INT;  
DECLARE rest INT;  
CALL intdiv(7, 3, NULL, NULL);  
-- d = 2, r = 1
```


Test av parameteroverføring

Parametertype angis i listen av formelle parametre

```
CREATE PROCEDURE param_test(  
    IN      p_1 INT,  
    OUT     p_2 INT,  
    INOUT   p_3 TEXT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    p_2 := p_1 + 1;  
    p_3 := p_3 || '...mer tekst.';  
END;  
$$;
```

Forsøk på å tilordne p_1 en verdi gir feilmelding

Bruk av SELECT for test-utskrift

```
DROP PROCEDURE IF EXISTS skriv2;
```

```
CREATE PROCEDURE skriv2()
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
DECLARE
```

```
    x INT;
```

```
BEGIN
```

```
    x := 2;
```

```
    RAISE NOTICE 'Tallet er: %', x;
```

```
END;
```

```
$$;
```

```
CALL skriv2();
```

```
Tallet er: 2
```

SELECT uten FROM er lov:

```
SELECT 2+2;
```

Nyttig for å teste ut funksjoner.

Lagret funksjon

```
DROP FUNCTION IF EXISTS minst(INT, INT);
```

```
CREATE FUNCTION minst(x INT, y INT)
```

```
RETURNS INT
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
    IF x < y THEN
```

```
        RETURN x;
```

```
    ELSE
```

```
        RETURN y;
```

```
    END IF;
```

```
END;
```

```
$$;
```

Funksjonskall for å teste:

```
SELECT minst(5,2);
```

SQL i lagrede rutiner

3 typer:

- Utvalgsspørringer som gir 1 rad (SELECT INTO)
- Utvalgsspørringer som kan gi flere rader – cursors
- Øvrige spørringer (INSERT, UPDATE, DELETE,...) enklest – skriv SQL rett inn i koden:

```
DROP PROCEDURE IF EXISTS store_bokstaver;  
CREATE PROCEDURE store_bokstaver()  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    UPDATE kunde  
    SET fornavn = UPPER(fornavn),  
        etternavn = UPPER(etternavn);  
END;  
$$;
```

SELECT INTO

Hvis en SQL-spørring alltid gir 1 rad kan verdiene kopieres over i lokale variabler vha **SELECT INTO**:

```
DECLARE
  v_pris NUMERIC(8, 2);
BEGIN
  SELECT pris_pr_enhet
  INTO v_pris
  FROM vare
  WHERE vnr = '10830';
END;
```

Vet at vi får 1 rad:

- Likheter
- vnr er primærnøkkel

Øvelse

Hvorfor og når brukes lagrede rutiner og funksjoner?

- **Bruksområder:**
 - Funksjoner brukes til å kjøre enkelte spørringer (hente data) og beregne en verdi (beregne alder eller total pris)
 - Prosedyrer brukes til å kjøre kompliserte spørringer (kombinasjon av select, insert, update og delete)
- **HVORFOR:** bedre ytelse, beskyttelse mot SQL injection angrep, kontroll over forretningslogikk

Forskjellen i formen mellom lagrede rutiner og funksjoner?

- Funksjoner tar inn parametre in paranteser og bruker RETURN til returverdi. Kalles ved bruk av «SELECT».
- Prosedyrer bruker IN, OUT og INOUT til å skille mellom inndata og returverdi. Kalles ved bruk av «CALL».

Cursors og spørreresultater

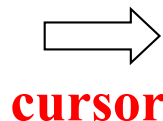
Cursor

- En peker som refererer en av radene i spørreresultatet.

For å behandle en databasetabell eller et spørreresultat:

- Ei løkke
- En «rad-peker» som flyttes
- Lese verdier i én og én rad

```
SELECT snr, navn,  
       adresse, fdato  
FROM student;
```



snr	navn	adresse	fdato
1	EVA	AVEIEN1	01.02.97
2	OLA	BVEIEN 2	10.02.03
5	KARI	AVEIEN 4	02.10.99
7	PER		02.10.92
9	GURI	CVEIEN 5	15.11.97

Bruk av cursors

```
DO $$
DECLARE
  c1 CURSOR FOR
    SELECT snr, navn
    FROM student;
  v_snr student.snr%TYPE;
  v_navn student.navn%TYPE;
BEGIN
  OPEN c1;

  LOOP
    FETCH c1 INTO v_snr, v_navn;
    EXIT WHEN NOT FOUND;

    RAISE NOTICE 'Snr: %, Navn: %', v_snr, v_navn;
  END LOOP;

  CLOSE c1;
END;
$$;
```


Øvelse

Query

Query History

1

SELECT * **FROM** fetch_film_titles_and_years();

Data Output

Messages

Notifications

<

```
CREATE OR REPLACE FUNCTION fetch_film_titles_and_years(  
    OUT p_title VARCHAR(255),  
    OUT p_release_year INTEGER  
)  
RETURNS SETOF RECORD AS  
$$  
DECLARE  
    film_cursor CURSOR FOR  
        SELECT title, release_year  
        FROM film;  
    film_record RECORD;  
BEGIN  
    -- Open cursor  
    OPEN film_cursor;  
  
    -- Fetch rows and return  
    LOOP  
        FETCH NEXT FROM film_cursor INTO film_record;  
        EXIT WHEN NOT FOUND;  
        p_title := film_record.title;  
        p_release_year := film_record.release_year;  
        RETURN NEXT;  
    END LOOP;  
  
    -- Close cursor  
    CLOSE film_cursor;  
END;  
$$  
LANGUAGE PLPGSQL;
```

Øvelse

Hvorfor og når brukes cursor?

Triggere

Databasehendelser

Avfyring av triggere

Rad-triggere

(Setnings-triggere)

BEFORE-triggere

AFTER-triggere

OLD og NEW

Bruksområder for triggere

A **database trigger** is procedural code that is **automatically executed** in response to certain **events** on a particular **table** [...] in a database. The trigger is mostly used for keeping the **integrity** of the information on the database.

[Wikipedia]

Hva er en trigger?

Et (lite) program som blir utført i databasen

- Kjøres automatisk som følge av en hendelse i databasen
- Mulige hendelser: INSERT, UPDATE, DELETE
- Vi sier at triggeren **avfyres**

Hvordan lage en trigger?

```
CREATE TRIGGER triggernavn
  [BEFORE | AFTER]
  [INSERT | UPDATE | DELETE] ON tabellnavn
  [FOR EACH ROW]
BEGIN
  setninger
END
```

Rad-trigger (1/2)

Blir avfyrt **for hver rad** som blir berørt av en oppdatering.

```
CREATE OR REPLACE FUNCTION
```

```
tr_ansatt_1_fn()
```

```
RETURNS trigger
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
-- Behandle raden
```

```
RETURN NEW;
```

```
END;
```

```
$$;
```

Rad-trigger (2/2)

```
DROP TRIGGER IF EXISTS tr_ansatt_1 ON  
ansatt;
```

```
CREATE TRIGGER tr_ansatt_1  
BEFORE UPDATE ON ansatt  
FOR EACH ROW  
EXECUTE FUNCTION tr_ansatt_1_fn();
```

Husk at SQL-setninger kan oppdatere mange rader.

```
UPDATE ansatt SET timeloenn = timeloenn * 1.1  
WHERE avdnr = 2;
```

(Setnings-triggere)

Blir avfyrt én gang for **hver SQL-setning**

– Setnings-triggere er støttet i PostgreSQL

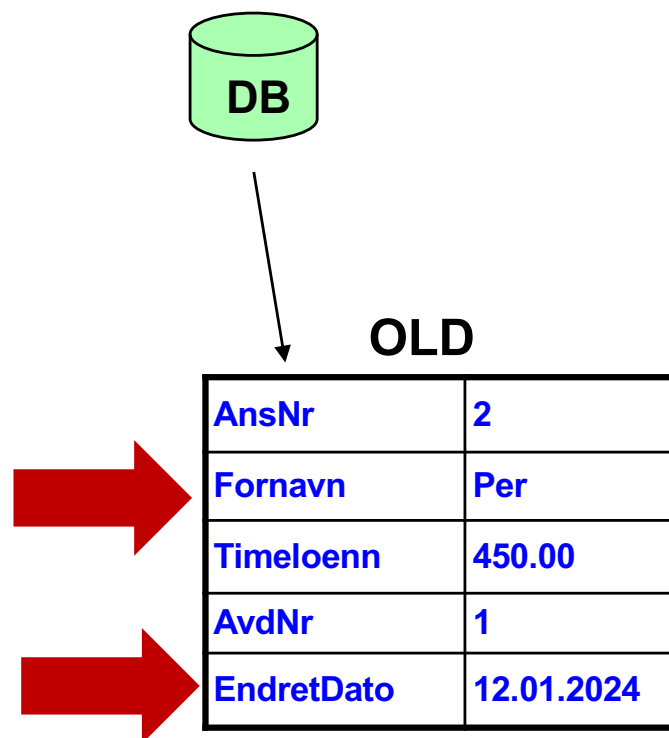
```
DROP TRIGGER IF EXISTS tr_ansatt_2 ON  
ansatt;
```

```
CREATE TRIGGER tr_ansatt_2  
BEFORE UPDATE ON ansatt  
EXECUTE FUNCTION tr_ansatt_2_fn();
```

Kan for eksempel brukes til aksesskontroll

Automatisk formatering – OLD og NEW (1/2)

```
CREATE OR REPLACE FUNCTION tr_ansatt_fn()  
RETURNS trigger  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    NEW.fornavn := UPPER(NEW.fornavn);  
    NEW.endretdato := CURRENT_DATE;  
  
    RETURN NEW;  
END;  
$$;
```



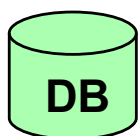
Automatisk formatering – OLD og NEW (2/2)

```
DROP TRIGGER IF EXISTS tr_ansatt ON ansatt;
```

```
CREATE TRIGGER tr_ansatt  
BEFORE UPDATE ON ansatt  
FOR EACH ROW  
EXECUTE FUNCTION tr_ansatt_fn();
```

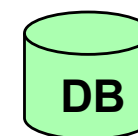
Kjører følgende oppdatering 27. februar 2025:

```
UPDATE Ansatt SET Fornavn = 'Petter'  
WHERE AnsNr = 2;
```



OLD

AnsNr	2
Fornavn	Per
Timeloenn	450.00
AvdNr	1
EndretDato	12.01.2024



NEW

AnsNr	2
Fornavn	PETTER
Timeloenn	450.00
AvdNr	1
EndretDato	27.02.2025

OLD og NEW

To innebygde «postvariabler» OLD og NEW

- OLD → opprinnelige verdier før endring
- NEW → nye verdier slik de **vil bli** etter oppdatering i databasen
- Ved INSERT er OLD = NULL, ved DELETE er NEW = NULL

Hvordan referere til data i raden som oppdateres

- OLD.<kolonnenavn> gir gammel verdi
- NEW.<kolonnenavn> gir ny verdi

BEFORE triggere → kjøres **før** databasen oppdateres

- NEW-verdier er endret og tilgjengelige selv om databasen ikke er oppdatert ennå!
- Triggeren kan endre NEW-verdier **før** oppdatering i basen
- Automatisk konvertering/formatering av data som settes inn
- Automatisk oppdaterte dataverdier, f.eks. datostempel som viser når raden sist ble endret

AFTER triggere → kjøres **etter** at databasen er oppdatert

- Da er det **for seint** å endre NEW-verdiene.
- OLD og NEW-verdier kan fremdeles **refereres**, f.eks. for å oppdatere andre tabeller

OLD og NEW

Hva inneholder bufferne?

	OLD	NEW
INSERT	NULL i alle kolonner	Verdier i den nye raden som skal settes inn
UPDATE	Verdier slik de er/var i databasen før oppdatering	Verdier slik de vil bli i databasen etter oppdatering
DELETE	Verdier slik de er/var i databasen før sletting	NULL i alle kolonner

OLD og NEW kan bare brukes i rad-triggere!

BEFORE - AFTER

Rad-triggere

	BEFORE	AFTER
INSERT	Ny rad er ikke satt inn ennå	Ny rad er satt inn i tabellen
UPDATE	Raden er ikke oppdatert i tabellen	NEW-verdier er oppdatert i tabellen
DELETE	Raden er ikke slettet ennå	Raden er slettet

Setnings-triggere

	BEFORE	AFTER
INSERT	Ingen rader er satt inn ennå	Alle nye rader er satt inn i tabellen
UPDATE	Ingen rader er oppdatert ennå	Alle berørte rader er oppdatert i tabellen
DELETE	Ingen rader er slettet ennå	Alle rader som berøres av DELETE er slettet fra tabellen

Loggføring og AFTER-triggere (1/2)

Hver gang noen endrer lønnen til en ansatt

- Sett inn en rad i en egen loggtabell **AnsattLogg**
- (Forutsetter at tabellen er opprettet)

```
CREATE OR REPLACE FUNCTION ansatt_a_upd_trg_fn()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO ansattlogg (ansnr, dato, gammellønn, nylønn)
    VALUES (
        NEW.ansnr,
        CURRENT_DATE,
        OLD.lønn,
        NEW.lønn
    );

    RETURN NEW;
END;
$$;
```

Loggføring og AFTER-triggere (2/2)

Hver gang noen endrer lønnen til en ansatt

- Sett inn en rad i en egen loggtabell **AnsattLogg**
- (Forutsetter at tabellen er opprettet)

DROP TRIGGER IF EXISTS ansatt_a_upd_trg ON ansatt;

CREATE TRIGGER ansatt_a_upd_trg

AFTER UPDATE ON ansatt

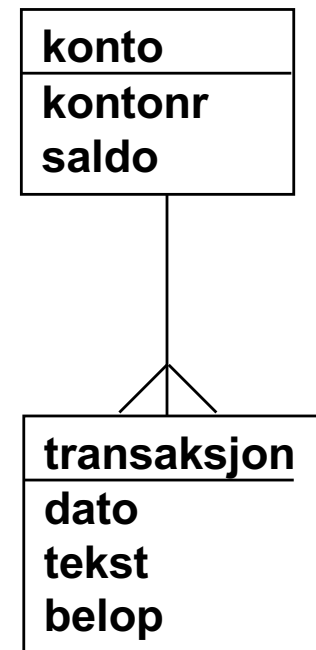
FOR EACH ROW

EXECUTE FUNCTION ansatt_a_upd_trg_fn();

Automatiske følgeoppdateringer (1/2)

Oppdatere saldo på konto etter en ny transaksjon

```
CREATE OR REPLACE FUNCTION tr_trans_ins_fn()  
RETURNS trigger  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    UPDATE konto  
    SET saldo = saldo + NEW.belop  
    WHERE kontonr = NEW.kontonr;  
  
    RETURN NEW;  
END;  
$$;
```

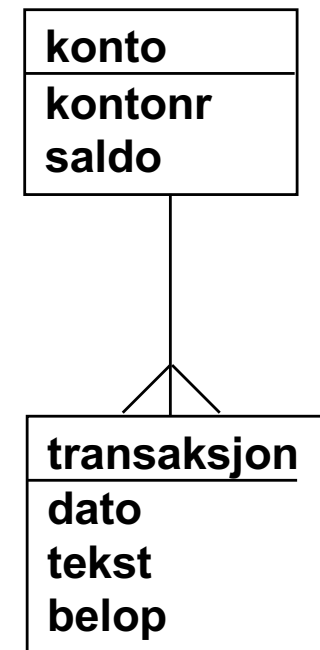


Automatiske følgeoppdateringer (2/2)

Oppdatere saldo på konto etter en ny transaksjon

```
DROP TRIGGER IF EXISTS tr_trans_ins ON transaksjon;
```

```
CREATE TRIGGER tr_trans_ins  
AFTER INSERT ON transaksjon  
FOR EACH ROW  
EXECUTE FUNCTION tr_trans_ins_fn();
```



Øvelse

Logging av endringer eksempel:

Gå til dvdrental databasen og opprett en tabell:

```
CREATE TABLE customer_log (  
  customer_id integer NOT NULL,  
  "O_email" character varying(50) NOT NULL,  
  "N_email" character varying(50) NOT NULL,  
  change_on timestamp with time zone NOT NULL,  
  PRIMARY KEY (customer_id)  
);
```

Opprett en trigger-funksjon:

```
CREATE OR REPLACE FUNCTION cust_email_changes()  
  RETURNS trigger  
  LANGUAGE plpgsql  
  AS $$  
  BEGIN  
    IF NEW.email <> OLD.email THEN  
      INSERT INTO customer_log VALUES  
        (OLD.customer_id,OLD.email,NEW.email,now());  
    END IF;  
    RETURN NEW;  
  END;  
  $$;
```

Opprett en trigger:

```
CREATE TRIGGER log_email_changes  
  BEFORE UPDATE  
  ON customer  
  FOR EACH ROW  
  EXECUTE PROCEDURE cust_email_changes();
```

Oppdater customer tabell:

```
UPDATE customer  
SET email = 'mary.smith@gmail.com'  
W  
HERE customer_id= 1;
```

Test endringene:

```
SELECT * FROM customer WHERE customer_id = 1;  
  
SELECT * FROM customer_log;
```

Forretningsregler (1/2)

CHECK-regler kan legges på i **CREATE TABLE** eller **ALTER TABLE**:

```
ALTER TABLE Vare  
ADD CHECK (Pris < 10000);
```

Men kan også bruke **trigger** i stedet:

```
CREATE OR REPLACE FUNCTION vare_b_upd_trg_fn()  
RETURNS trigger  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF NOT (NEW.pris < 10000) THEN  
        RAISE EXCEPTION 'For dyr vare!';  
    END IF;  
  
    RETURN NEW;  
END;  
$$;
```

Forretningsregler (2/2)

```
DROP TRIGGER IF EXISTS vare_b_upd_trg ON vare;
```

```
CREATE TRIGGER vare_b_upd_trg  
BEFORE UPDATE ON vare  
FOR EACH ROW  
EXECUTE FUNCTION vare_b_upd_trg_fn();
```

Beskrivelse av triggere i systemkatalogen (1/2)

Metadata-kommando:

```
SELECT
    tg.tgname          AS trigger_name,
    tbl.relname        AS table_name,
    tg.tgenabled       AS enabled,
    pg_get_triggerdef(tg.oid, true) AS definition
FROM pg_trigger tg
JOIN pg_class tbl ON tbl.oid = tg.tgrelid
JOIN pg_namespace ns ON ns.oid = tbl.relnamespace
WHERE NOT tg.tgisinternal
ORDER BY table_name, trigger_name;
```

SQL-spørring om triggere på en spesifikk tabell:

```
SELECT tgname FROM pg_trigger, pg_class WHERE
    tgrelid=pg_class.oid AND relname='customer';
```

Beskrivelse av triggere i systemkatalogen (2/2)

PostgreSQL: Eller spør mot metatabellen **information_schema.triggers**:

```
SELECT
    trigger_schema,
    trigger_name,
    event_manipulation,
    event_object_table,
    action_statement,
    action_timing
FROM information_schema.triggers;
```

Lag en SQL-spørring som **lager** et SQL-skript som sletter triggere:

```
DROP TRIGGER IF EXISTS trigger_name ON table_name;
```

Bruksområder for triggere

BEFORE triggere – fyres **før** oppdatering i basen

- Automatisk formatering av verdier
- Automatisk datostempling av rader
- Kontroll med forretningsregler
- Finmasket tilgangskontroll

AFTER triggere - fyres **etter** oppdatering i basen

- Automatiske oppdateringer i andre tabeller
- Loggføring av oppdateringer
- Oppdatere avledete data i andre tabeller

Brukes for operasjoner som skal gjøres «uten unntak»!